

# CS 344 Lab-1 Submission Report

Mit Narodia (230123039)  
Rehan Sherawat (230123077)  
Samir Raj (230123056)  
Prakhar Agrawal (230123045)

August 4, 2025

## 1 Task 1.1: Prime PID Assignment

### 1.1 Results:

```
xv6 kernel is booting  
  
Allocated PID: 2  
hart 1 starting  
hart 2 starting  
init: starting sh  
Allocated PID: 3  
$ forktest  
Allocated PID: 5  
fork test  
Allocated PID: 7  
Allocated PID: 11  
Allocated PID: 13  
Allocated PID: 17  
Allocated PID: 19  
Allocated PID: 23  
Allocated PID: 29  
Allocated PID: 31  
Allocated PID: 37  
Allocated PID: 41  
Allocated PID: 43  
Allocated PID: 47  
Allocated PID: 53  
Allocated PID: 59  
Allocated PID: 61  
Allocated PID: 67  
Allocated PID: 71  
Allocated PID: 73  
Allocated PID: 79  
Allocated PID: 83  
Allocated PID: 89  
Allocated PID: 97  
Allocated PID: 101  
Allocated PID: 103  
Allocated PID: 107  
Allocated PID: 109  
Allocated PID: 113  
Allocated PID: 127  
Allocated PID: 131  
Allocated PID: 137  
Allocated PID: 139  
Allocated PID: 149  
Allocated PID: 151
```

Figure 1: All PIDs are consecutive primes.

## 1.2 Objective

The goal of this task was to modify the standard PID allocation logic in xv6. Instead of assigning sequential integers (1, 2, 3, ...), the new logic must assign the next available prime number (2, 3, 5, 7, ...) to each new process created by the system.

## 1.3 Implementation: kernel/proc.c

The core of this task was modifying the `allocpid()` function and adding a helper function within `kernel/proc.c`.

### 1.3.1 Helper Function: `is_prime()`

A helper function was necessary to determine if a given integer is a prime number.

**Code Modification** The following function was added to `kernel/proc.c`.

```
int is_prime(int n){
    for(int i = 2 ; i*i <= n ; i++){
        if(n%i == 0)return 0 ;
    }
    return 1 ;
}
```

**Justification:** This helper function checks whether a number is prime using a simple trial division method. Adding it makes the code in `allocpid()` cleaner and easier to read by separating the logic of checking primality. Since this check is used repeatedly during PID assignment, it's better to keep it as a separate function for clarity and potential reuse.

### 1.3.2 PID Allocation Logic: `allocpid()`

The existing `allocpid()` function was replaced with a version that finds and assigns the next prime number.

**Code Modification** The `allocpid()` function in `kernel/proc.c` was changed to the following:

```
int
allocpid()
{
    int pid;
    acquire(&pid_lock);

    pid=nextpid;
    nextpid++;
    while(!is_prime(nextpid)) nextpid++;
    printf("Allocated PID: %d\n", pid);
    release(&pid_lock);

    return pid;
}
```

**Justification:** We changed `allocpid()` so that it now finds the next prime number starting from the last assigned PID. This ensures that all process IDs are unique and prime, as required. The while loop keeps increasing the value until a prime is found. We also made sure to use `pid_lock` while updating the PID to prevent race conditions when multiple processes are being created at the same time. This makes the PID assignment both correct and thread-safe.

## 2 Task 1.2: Implement top Command

### 2.1 Objective

The objective of this task was to create a user-space program named `top` that lists all currently running processes and displays their Process ID (PID), Process State, Number of Ticks (CPU time), and Process Name.

### 2.2 Results:

```
xv6 kernel is booting

Allocated PID: 2
hart 2 starting
hart 1 starting
init: starting sh
Allocated PID: 3
$ top
Allocated PID: 5
PID      STATE      TICKS    NAME
2        SLEEPING    31      init
3        SLEEPING    13      sh
5        RUNNING  6        top
$ zombie_test &
Allocated PID: 7
Allocated PID: 11
$ Allocated PID: 13
top
Allocated PID: 17
PID      STATE      TICKS    NAME
2        SLEEPING    31      init
3        SLEEPING    21      sh
13       ZOMBIE    9        zombie_test
11       RUNNING  37       zombie_test
17       RUNNING  2        top
$ top
Allocated PID: 19
PID      STATE      TICKS    NAME
2        SLEEPING    31      init
3        SLEEPING    23      sh
13       ZOMBIE    9        zombie_test
11       RUNNING  182      zombie_test
19       RUNNING  0        top
$ top
Allocated PID: 23
PID      STATE      TICKS    NAME
2        SLEEPING    31      init
3        SLEEPING    25      sh
23       RUNNING  0        top
$
```

Figure 2: Task two results.

### 2.3 Explanation of Results

The `zombie_test` program demonstrates the creation of a zombie process in a controlled environment. The program performs the following steps:

1. It calls `fork()` to create a child process. The child process immediately exits after simulating some work.
2. The parent process, while still alive, does not wait for the child process to exit by calling `wait()`. This prevents the child process from being cleaned up and causes it to remain in the process table as a zombie.

3. The parent process continues running for a significant amount of time before exiting, which leaves the child process in a zombie state.

As shown in the `top` command's output (Figure 2), the child process is displayed in the **ZOMBIE** state. The process ID (PID) of the zombie process remains in the process table until the parent terminates. This is a clear demonstration of how zombie processes behave in a Unix-like operating system.

## 2.4 Kernel Data Structure Modification (kernel/proc.h)

**Code Modification** 1. The `ticks` field was added to `struct proc` to track CPU time. 2. A new Data Transfer Object, `struct proc_info`, was created.

```
// In struct proc:
struct proc {
    // ... existing fields
    int pid;                // Process ID
    int ticks;              // CPU time consumed by the process
    // ... rest of struct
};

// struct for top command
struct proc_info {
    int pid;
    int state;
    int ticks;
    char name[16];
};
```

**Justification** We added a new field `ticks` to the `proc` struct to keep track of how much CPU time each process has used. This is useful for monitoring process activity and for tools like `top` that display CPU usage. The new `proc_info` struct was added as a lightweight version of the process structure. It only contains the fields we want to expose to user space. This avoids sharing the full kernel-level structure with user programs, which could be unsafe or unnecessary.

## 2.5 Kernel Logic Implementation (proc.c & sysproc.c)

### 2.5.1 kernel/proc.c

**Code Modification** The logic for managing ticks and gathering process info was added.

```
// In allocproc()
static struct proc*
allocproc(void)
{
    // ...
    p->pid = allocpid();
    p->state = USED;
    p->ticks = 0;
    // ...
}

// In scheduler()
void
scheduler(void)
{
    // ...
    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == RUNNABLE) {
            // ...
            c->proc = 0;
        } else if(p->state == RUNNING){
```

```

        p->ticks++;
    }
    release(&p->lock);
}
// ...
}

// New function
extern struct proc proc[NPROC]; //(to reference this element in another file)

int getprocsinfo(struct proc_info *infos, int max)
{
    int count = 0;

    acquire(&proc_lock);
    for (int i = 0; i < NPROC && count < max; i++) {
        if (proc[i].state != UNUSED) {
            infos[count].pid = proc[i].pid;
            infos[count].state = proc[i].state;
            infos[count].ticks = proc[i].ticks;
            safestrcpy(infos[count].name, proc[i].name, sizeof(proc[i].name));
            count++;
        }
    }
    release(&proc_lock);

    return count;
}

```

**Justification:** In `allocproc()`, we initialized ticks to 0 so that every new process starts with zero CPU usage. In `scheduler()`, we increment the ticks of a process every time it's in the RUNNING state. This gives a rough measure of how much CPU time each process has used. The `getprocsinfo()` function collects basic information (PID, state, ticks, name) about all processes that are not UNUSED. It fills this into an array and returns the count. We use a lock to make sure the process list isn't changed while we're reading from it. This is important for consistency and correctness.

### 2.5.2 kernel/sysproc.c

**Code Modification** A new system call handler was created.

```

uint64
sys_getprocsinfo(void)
{
    uint64 uinfos_addr;          // temporary to hold the address passed from user
    int max;

    argaddr(0, &uinfos_addr);
    argint(1, &max);
    struct proc_info kinfos[MAX_PROCS];
    int count = getprocsinfo(kinfos, max);
    if(count < 0) return count;
    struct proc *p = myproc();
    if(copyout(p->pagetable, uinfos_addr, (char*)kinfos, count*sizeof(struct proc_info)) < 0)
        return -1;
    return count;
}

```

**Justification:** This function is the system call handler that bridges the kernel and user space. It receives a user-provided memory address and maximum number of entries to return. It gathers the

data in a local array, then uses `copyout()` to safely transfer that array to user memory. Using `copyout()` prevents direct access to kernel memory and ensures memory protection is maintained. This design makes it safe and efficient to expose process info to user programs.

## 2.6 System Call Registration (`syscall.h` & `syscall.c`)

**Code Modification** (`kernel/syscall.h`)

```
#define SYS_getprocsinfo 23
```

**Code Modification** (`kernel/syscall.c`)

```
extern uint64 sys_getprocsinfo(void);

static uint64 (*syscalls[])(void) = {
    //...
    [SYS_getprocsinfo]    sys_getprocsinfo,
};
```

**Justification:** In `syscall.h`, we gave the new system call a unique number. In `syscall.c`, we connected that number to the handler function (`sys_getprocsinfo`). This step is necessary so that when a user program makes the system call, the kernel knows which function to execute. Without this mapping, the system call wouldn't be recognized by the kernel.

## 2.7 User-Space Integration (`user.h` & `usys.pl`)

**Code Modification** (`user/user.h`)

```
struct proc_info {
    int pid;
    int state;
    int ticks;
    char name[16];
};

int getprocsinfo(struct proc_info*, int);
```

**Code Modification** (`user/usys.pl`)

```
entry("getprocsinfo");
```

**Justification** We declared the `proc_info` struct and the `getprocsinfo()` function prototype in `user.h` so that user programs know how to call it and what the expected input/output is. In `usys.pl`, we added an `entry()` for `getprocsinfo`, which generates the actual assembly code required to switch from user mode to kernel mode. This is essential for making the system call work from C code in user space.

## 2.8 The top Application (`user/top.c`)

A new file, `user/top.c`, was created.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

#define MAX_PROCS 64

char *states[] = {
    "UNUSED", "USED", "SLEEPING", "RUNNABLE", "RUNNING", "ZOMBIE"
};

int
main() {
```

```

struct proc_info infos[MAX_PROCS];
int n = getprocsinfo(infos, MAX_PROCS);
printf("PID\tSTATE\t\tTICKS\tNAME\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%s\t%d\t%s\n", infos[i].pid,
           states[infos[i].state], infos[i].ticks, infos[i].name);
}

exit(0);
}

```

**Justification:** The top command is a user-level program that calls the `getprocsinfo()` system call, gets data about running processes, and displays it in a readable table. It maps numeric states (like `RUNNING`, `SLEEPING`, etc.) to their string versions to make the output more understandable. This program helps users monitor what processes are active, how much CPU time they’ve used, and in what state they are. It’s especially helpful for debugging or observing system behavior in real time.

## 2.9 The zombie test Application (user/zombie\_test.c)

A new file, `user/zombie_test.c`, was created.

```

#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int main() {
    int pid = fork();

    if (pid < 0) {
        printf("fork failed\n");
        exit(1);
    }

    if (pid == 0) {
        // Child: exit immediately
        for(long long i = 0 ; i < 1000000000;i++);
        exit(0);
    }

    // Parent: sleep without calling wait
    for(long long i = 0 ; i < 1000000000;i++); // enough time to run 'top' and see zombie
    exit(0);
}

```

## 2.10 Build System Integration (Makefile)

### Code Modification

```

UPROGS=\
...
$U/_top\
$U/_zombie_test

```

**Justification** We added `_top` to the `UPROGS` variable in the Makefile so that the top program gets compiled and included in the xv6 image. This makes it available to run inside the xv6 shell like any other command. We also added `_zombie_test`, which is a small helper program used to test how top behaves with zombie or sleeping processes.

### 3 Conclusion

In Task 1.1, we learned how to change a core part of the system — the process ID allocation — while making sure it works safely in a multi-process environment using locks and helper functions. It showed us how small changes at the kernel level can impact how the entire system behaves.

In Task 1.2, we implemented a complete feature from the kernel to user space. We learned how to expose internal process data through a secure system call and how to build a user-facing program (top) that displays this information in a meaningful way. This task helped us understand system calls, data handling between kernel and user space, and how to design lightweight data structures for communication.

### References

- [1] Russ Cox, Frans Kaashoek, and Robert Morris. *Xv6, a simple, Unix-like teaching operating system*.
- [2] YouTube. (2018). *System Call Implementation in xv6*. Retrieved from <https://www.youtube.com/watch?v=QtZ7PtetIic>.
- [3] Mythili Vutukuru. *CS 347: Operating Systems, IIT Bombay*. Retrieved from <https://www.cse.iitb.ac.in/~mythili/os/>.